

Reviewing documents on GitHub

A guide for reviewers and authors

Documentation Team

2026-10-07

Table of Contents

Why we review this way.....	3
The process at a glance	3
The worked example	3
For reviewers.....	5
How a review is requested	5
Automatic reviewers with CODEOWNERS	5
How the file works	5
Example.....	6
Changing the reviewers.....	6
1. Open the Pull Request	7
2. Read the document	7
3. Comment on a line	8
4. Comment on several lines	9
5. Suggest new wording	10
6. Batch your comments with “Start a review”	11
7. Submit your review with the right verdict.....	12
8. Re-review only what changed.....	14
For authors	15
1. Create a branch	15
2. Write one sentence per line.....	15
3. Open the Pull Request with the template	15
4. Full review vs partial review.....	16
5. Answer comments	17
6. Commit suggestions.....	18
7. Make your other fixes.....	19
8. Resolve the conversations	19

9. Merge	22
10. Get the published files	23
FAQ	25
Cheat sheet	26

Why we review this way

Our documents are written in Markdown, stored in GitHub, and reviewed through Pull Requests (PRs) instead of passing Word files around by email or sharing copies on SharePoint sites.

- **One source of truth.** There is one copy of each document, in the repository. There are no “v3_final_JM_reviewed.docx” files.
- **Comments are tied to exact sentences.** Each comment is attached to the line it talks about, so nobody has to guess what “the second paragraph on page 4” means.
- **Suggestions work like tracked changes.** A reviewer can propose new wording, and the author accepts it with one click.
- **Full history.** GitHub records who proposed, reviewed, approved and published every change, and when.
- **The published files always match the approved text.** The Word file (DOCX) is generated automatically from the approved Markdown, never edited by hand.
- **You don’t need to know Git.** Reviewers and authors only use the GitHub website.
- **Everything happens in the browser.** You don’t need to install anything.

The process at a glance

1. The author writes or changes a document on a separate branch and opens a Pull Request.
2. GitHub builds the Word file (DOCX) for that version automatically.
3. Reviewers read the document, comment on specific lines, and propose wording.
4. Reviewers submit their verdict: **Comment**, **Request changes**, or **Approve**.
5. The author answers, applies suggestions, fixes the rest, and resolves each conversation.
6. The reviewer checks only what changed and approves.
7. The author merges. GitHub builds the final Word file from main.

main is protected. Nothing can be merged without at least one approval and with every conversation resolved.

The worked example

All screenshots in this guide come from a real review in this repository: [Pull Request #1](#), “Add sample Information Security Policy”.

- **Author:** jmizquierdo
- **Reviewer:** heydolon
- **Document:** docs/sample-policy.md

The document had four planted mistakes:

- the typo “anually”;
- a vague rule (“changed regularly”);
- an empty cell in a table;
- critical incidents with a longer deadline (48 hours) than normal ones (24 hours).

You can open the PR and follow along.

For reviewers

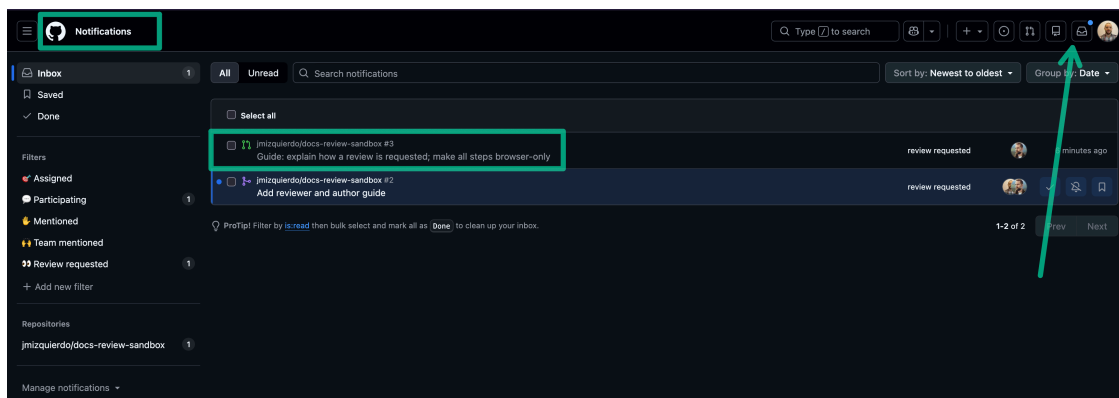
How a review is requested

A review starts when the author asks you for one. The reviewer must already be a **collaborator** on the repo (Settings → Collaborators → **Add people**, and they accept the invitation), or they won't appear in the list.

The author requests the review in one of two places:

- **When you open the PR:** on the “Open a pull request” form, click the gear next to **Reviewers** in the right column, type the username, select it, then click **Create pull request**.
- **On a PR that's already open:** on the PR's **Conversation** tab, click the gear next to **Reviewers** in the right column and select the user. If they reviewed before, the circular-arrows icon next to their name asks them to review again.

The reviewer gets a notification and an email, and the PR appears under **Pull requests** → **Review requests** for them.



A review request in the reviewer's notifications inbox (bell icon, top right)

Automatic reviewers with CODEOWNERS

Instead of picking reviewers by hand every time, the repository can request them automatically. The CODEOWNERS file, at the top level of the repository, says who reviews which files. When a PR changes a file, GitHub requests a review from that file's owners as soon as the PR is opened. The author can still add more reviewers by hand.

In this repository the file contains one line, so heydo1on is requested on every PR:

```
* @heydo1on
```

How the file works

- Each line has a **file pattern** followed by one or more **owners**, separated by spaces.

- An owner is a GitHub username with @ (@heydolon), a team in an organization (@my-org/docs-team), or the email address of a GitHub account.
- * means every file. A folder ends with / (/docs/). A single file is written with its full path (/docs/sample-policy.md).
- Lines that start with # are comments.
- **The last matching line wins.** Put the general rules first and the specific ones after them.
- Owners must be collaborators with write access. GitHub ignores owners it can't use, and shows a warning at the top of the file when you open it on GitHub.
- The author of a PR is never asked to review their own PR, even if they own the file.

Example

```
# Default: heydolon reviews everything
*                               @heydolon

# Every document in docs/ is reviewed by the documentation team
/docs/                          @my-org/docs-team

# The security policy needs two people: both are requested
/docs/sample-policy.md         @heydolon @alice

# The guide and its screenshots
/REVIEWING.md                  @heydolon
/guide/images/                 @heydolon

# Changes to the build go to whoever maintains it
/.github/                      @bob
```

With this file:

- A PR that changes docs/password-policy.md requests @my-org/docs-team. The /docs/ line is the last one that matches.
- A PR that changes docs/sample-policy.md requests @heydolon and @alice, but not the team. The more specific line comes later, so it wins.
- A PR that changes README.md requests @heydolon. Only the * line matches.

Changing the reviewers

CODEOWNERS is changed like any document, through a PR:

1. Create a branch (see “For authors”, step 1).
2. On your branch, open CODEOWNERS and click the pencil icon.
3. Add, remove or change lines, for example add @alice after @heydolon to make both review everything.
4. **Commit changes** to your branch and open a PR. The current owners are asked to approve the change.
5. The new rules apply to PRs opened **after** the change is merged into main.

Optionally, the repository admin can switch on **Require review from Code Owners** in the branch protection of main. Then a PR can't be merged until one of the owners of each changed file has approved it. It is not switched on in this repository.

1. Open the Pull Request

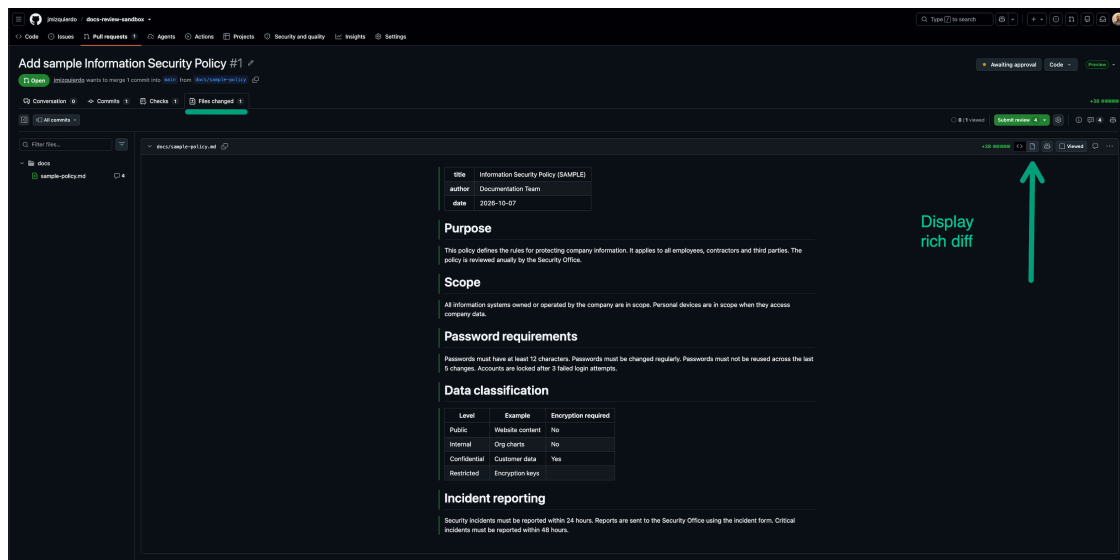
You get a notification (email and the bell icon on GitHub) when someone asks for your review. You can also go to **Pull requests** in the top bar and choose **Review requests**.

Open the PR. The **Conversation** tab shows the description written by the author: which documents are under review, whether it is a full or partial review, what changed, and the deadline.

2. Read the document

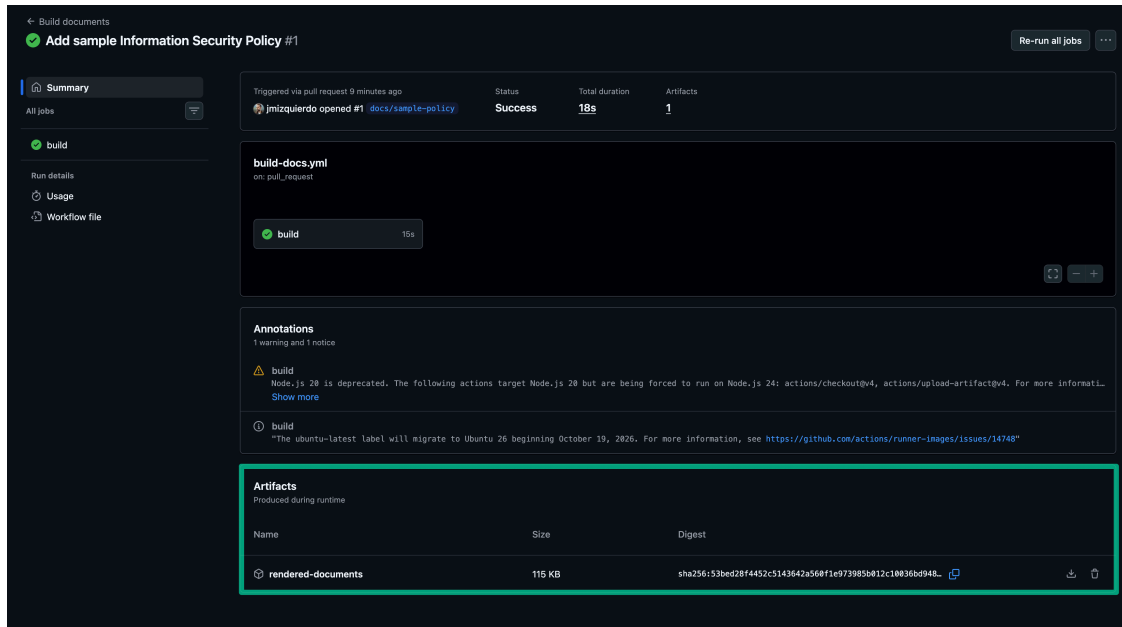
There are two ways to read the document.

In the browser. Open the **Files changed** tab. In the header of the file, click the **Display the rich diff** icon (the page icon) to see the formatted document instead of the raw text.



The rendered (“rich diff”) view of the document in Files changed

As a Word file. Open the **Actions** tab, choose **Build documents**, and download rendered-documents from the **Artifacts** section. It is a ZIP file with the DOCX of exactly this version. To read it as a PDF, open it in Word and save it as PDF.



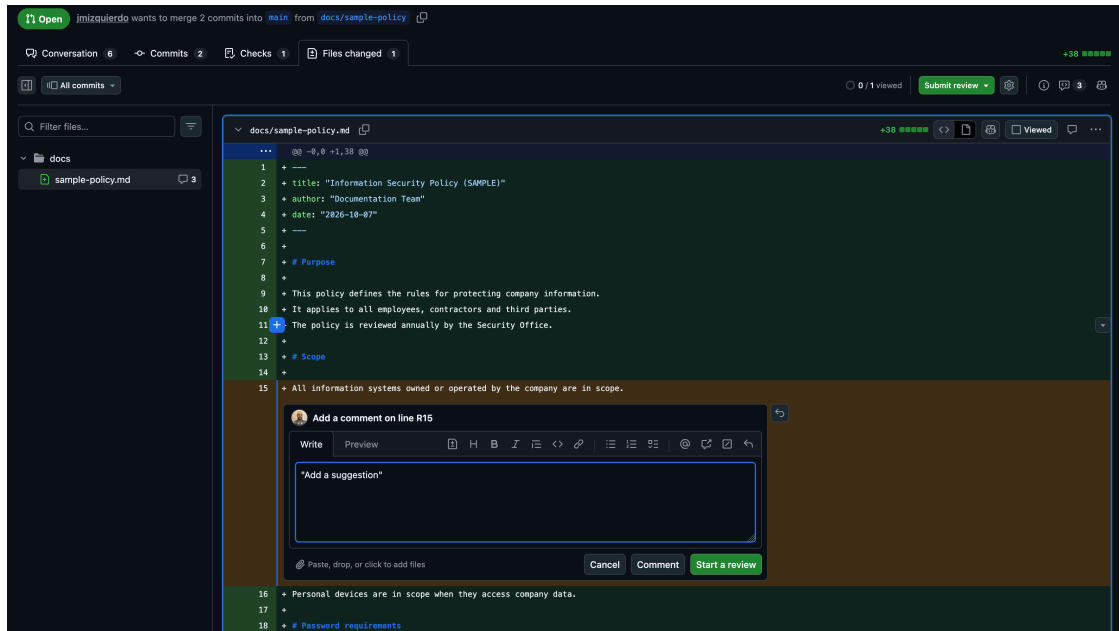
The rendered-documents artifact on the build page

You can read the Word file, but **write your comments on GitHub**, not in the file. Comments in a downloaded file are lost.

3. Comment on a line

Go back to the source view in **Files changed** (the <> icon in the file header). Each sentence is on its own line, with a line number on the left.

Hover over the line number and click the blue + that appears. A comment box opens under the line.



A comment box opened on a single line

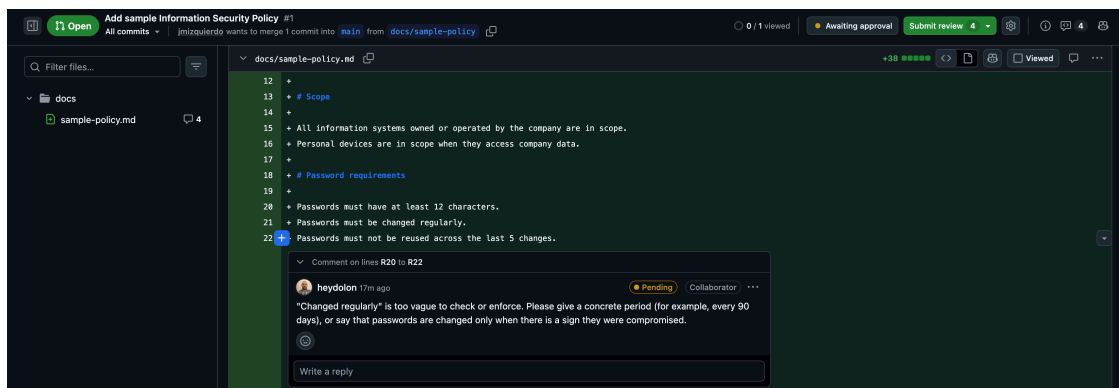
Write your comment and click **Start a review** (see step 6).

If you can't see the +, you are probably still in the rendered view. Switch back to the source view.

4. Comment on several lines

Click the + on the first line and drag down to the last line, or click the first line number and Shift-click the last one. The comment then covers the whole block, and its header says “Comment on lines +20 to +22”.

In the example, the reviewer selected the three password rules to say “changed regularly” was too vague.



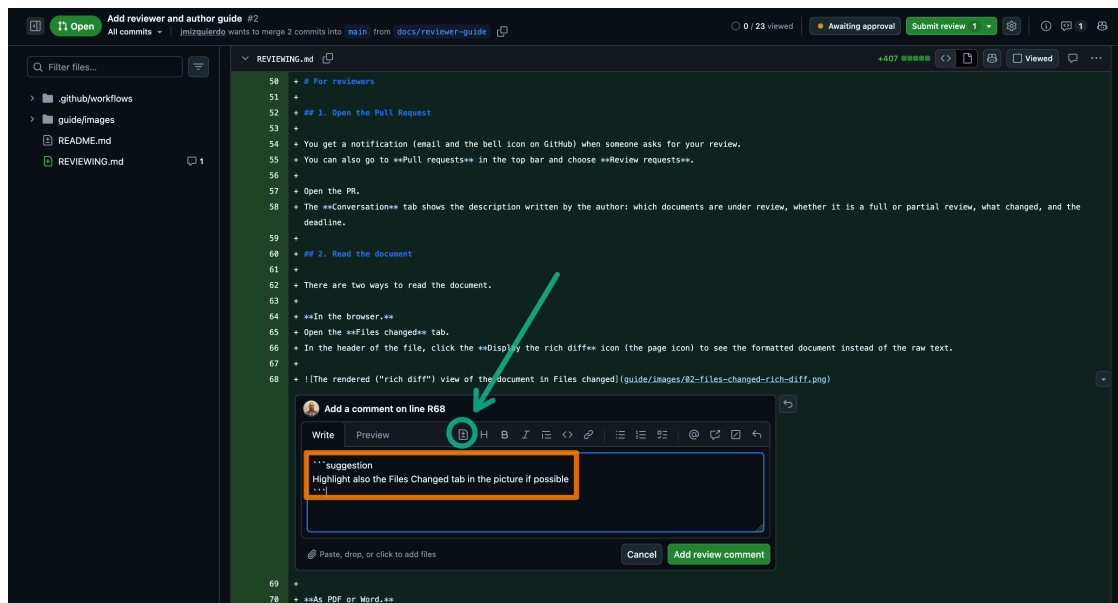
A comment covering lines 20 to 22

5. Suggest new wording

When you know exactly what the text should say, propose it instead of describing it.

1. Open a comment box on the line, as in step 3 (or on several lines, as in step 4).
2. Click the first icon in the comment toolbar, the document with a plus and minus sign (**Add a suggestion**). On a Mac you can also press Cmd+G.
3. GitHub inserts a block that starts with `` `` suggestion` and contains the current text.
4. Edit the text **inside** the block so it reads the way it should.
5. Optionally, add a short explanation **above** the block.

Only the new text goes inside the block. Whatever is inside replaces the original line(s) when the author commits the suggestion. If you write an instruction there, such as “Highlight the tab in the picture”, committing it would replace the line with your instruction. In this screenshot from the review of this guide, committing the suggestion would have deleted the image:



Wrong: an instruction written inside a suggestion block

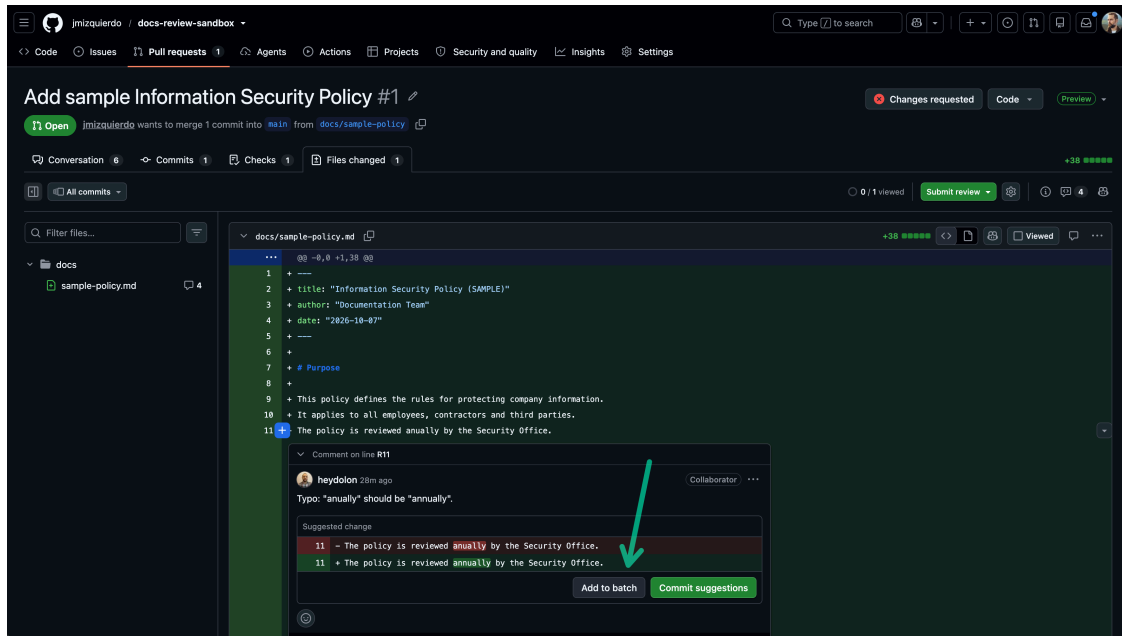
For requests like that, write a normal comment without a suggestion block.

In the example, the reviewer fixed the typo like this:

Typo: "anually" should be "annually".

```
` `` suggestion
The policy is reviewed annually by the Security Office.
` ``
```

The author sees the old and new versions side by side and can accept the change with one click.

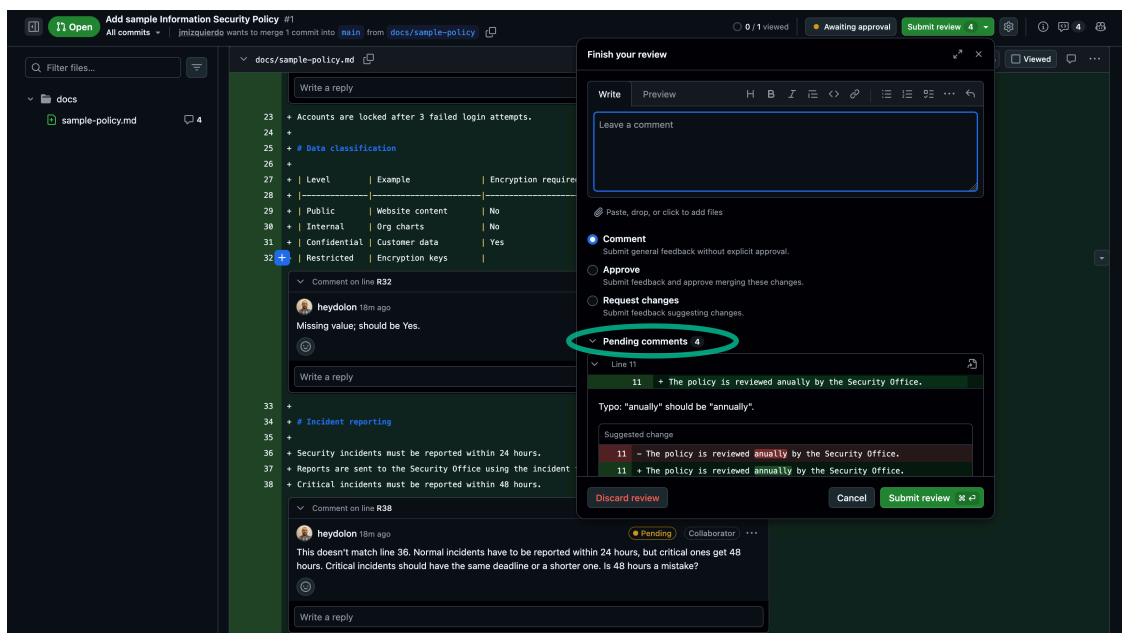


The suggestion as the author sees it, with the buttons to accept it

Use suggestions for wording, typos and small fixes. Use a normal comment when you are asking a question or the fix needs the author's judgement.

6. Batch your comments with “Start a review”

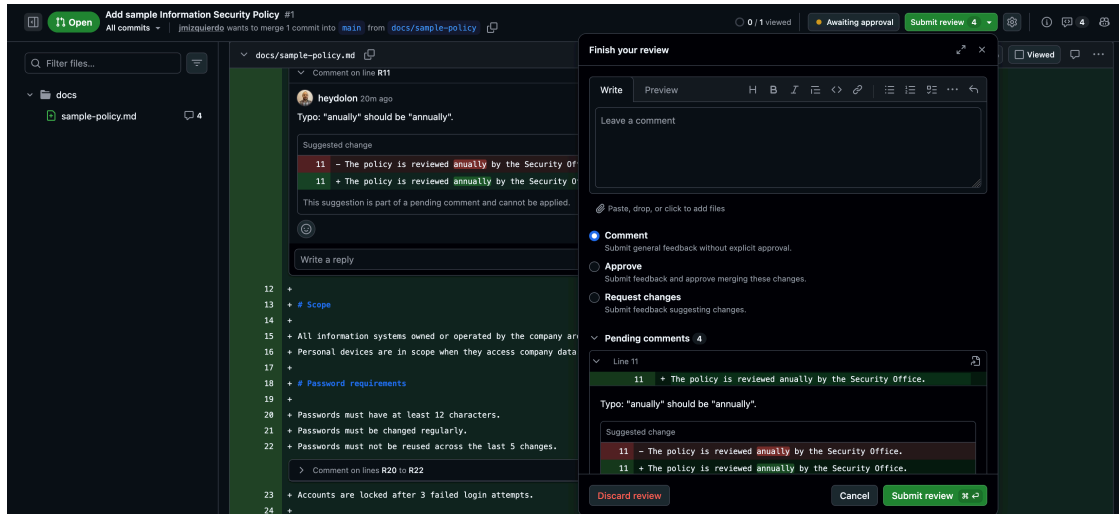
When you write your first comment, click **Start a review** instead of **Add single comment**. Your comments stay **Pending**, visible only to you, until you submit them all together. This way the author gets one notification with all your feedback, not one per comment.



Pending comments, and the pending count on the Submit review button

7. Submit your review with the right verdict

When you are done, click **Submit review** (called **Review changes** on some pages) in the top right. Write a short summary and choose a verdict.



The Finish your review dialog with the three verdicts

Verdict	Use it when	Effect
Comment	You have questions or minor remarks and don't want to block or approve.	Doesn't change whether the PR can be merged.
Request changes	Something must be fixed before publishing.	Blocks the merge until you approve.
Approve	The document is ready to publish.	Counts as the approval needed to merge.

In the example, the reviewer submitted four comments with **Request changes**. The PR then showed the review and its threads on the **Conversation** tab, and merging was blocked.

Open **Add sample Information Security Policy #1**
jmitzquierdo wants to merge 1 commit into `main` from `docs/sample-policy`

Conversation 6 Commits 1 Checks 1 Files changed 1

jmitzquierdo commented 25 minutes ago

Document(s) under review

- docs/sample-policy.md: Information Security Policy (SAMPLE)

Type of review

- ☒ Full review of a new document
- ☐ Partial review of changes to an existing document

Summary of changes

First draft of the sample Information Security Policy. It covers the purpose, scope, password requirements, data classification and incident reporting. It is written one sentence per line, so each sentence can be commented on separately.

Reviewer instructions

- Read the document. In the Files changed tab, click the file's -- menu → View file, or use the Display the rich diff button (page icon) to see the rendered text. Or download the PDF/DOCX from the Checks tab → Build documents → Artifacts → rendered-documents.
- Comment on specific lines. Hover over a line in Files changed and click the blue +. Drag the + over several lines to comment on a block.

Reviewers

- heydoton
Requested changes must be addressed to merge this pull request.

Still in progress? Convert to draft

Assignees

No one—assign yourself

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

None yet

The submitted review and its threads on the Conversation tab

Open **Add sample Information Security Policy #1**
jmitzquierdo wants to merge 1 commit into `main` from `docs/sample-policy`

Changes requested
1 change requested by reviewers with write access.

1 requested change >

All checks have passed
1 successful check >

Merging is blocked
All comments must be resolved.
1 review requesting changes by reviewers with write access.

Merge pull request You can also merge this with the command line. [View command line instructions.](#)

Still in progress? [Convert to draft](#)

Add a comment

Write Preview

Add your comment here...

Markdown is supported Paste, drop, or click to add files

Close pull request Comment

Remember, contributions to this repository should follow our [GitHub Community Guidelines](#).

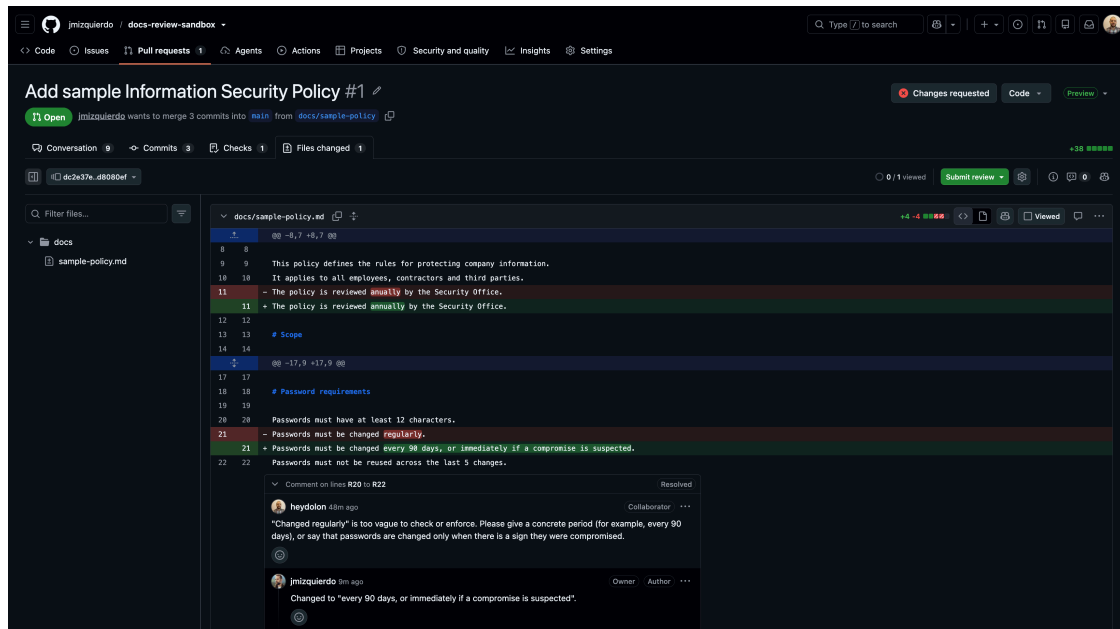
ProTip! Add comments to specific lines under [Files changed](#).

Merging is blocked because changes were requested

8. Re-review only what changed

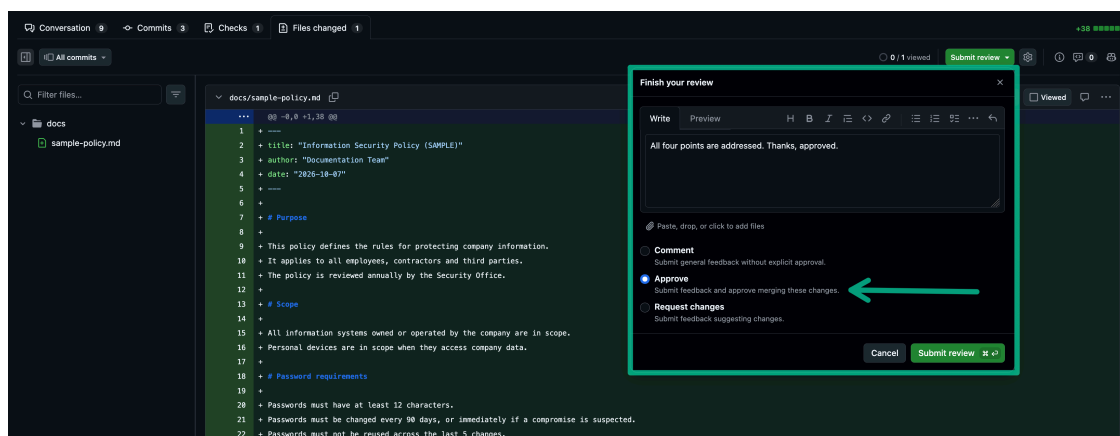
When the author commits fixes, you get a new notification. You don't need to read the whole document again.

In **Files changed**, open the **All commits** menu at the top left and choose **Changes since your last review**. You see only the lines the author changed after your review.



Only the changes since the last review

Check that every point was addressed, and look at the author's replies on the **Conversation** tab. Then submit a new review. If everything is fine, choose **Approve**.



Approving the PR

Your previous “Request changes” verdict stays in place until **you** approve. The author fixing everything is not enough to unblock the merge.

For authors

1. Create a branch

Never edit **main** directly; it is protected. Each change, or each new document, gets its own branch with a descriptive name, for example `docs/sample-policy` or `docs/password-policy-update`.

1. On the **Code** tab of the repository, click the branch menu at the top left of the file list (it shows **main**).
2. Type the new branch name and click **Create branch docs/my-change from main**.
3. GitHub switches to your branch. Check that the branch menu now shows its name.

To work on your branch:

- **Change a document:** open the file and click the pencil icon (**Edit this file**).
- **Add a document:** click **Add file**, then **Create new file**, and type the name with its folder, for example `docs/password-policy.md`.

When you are done, click **Commit changes**, write a short description, and choose **Commit directly to the docs/my-change branch**.

If you start editing on **main** by mistake, GitHub won't let you commit there and offers **Create a new branch for this commit and start a pull request** instead, which is fine too.

2. Write one sentence per line

Put each sentence on its own line. Markdown joins the lines of a paragraph when the document is rendered, so the Word file looks exactly the same.

```
Security incidents must be reported within 24 hours.  
Reports are sent to the Security Office using the incident form.
```

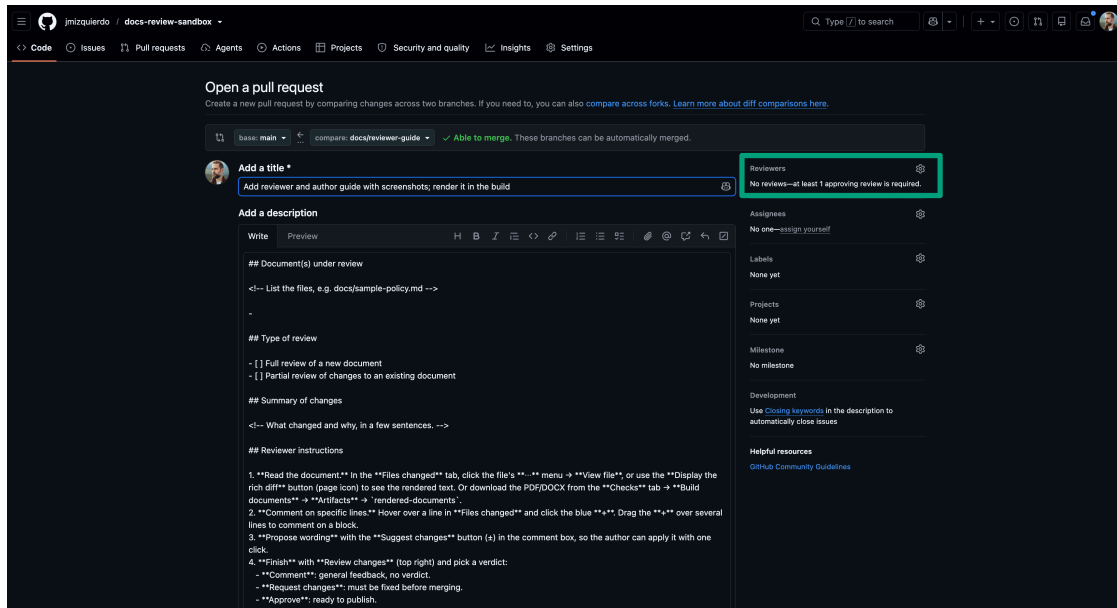
Why it matters:

- Reviewers can comment on **one sentence** instead of a whole paragraph.
- Suggestions replace one sentence and don't touch the others.
- When you change a sentence, the diff shows that sentence only.

Leave an empty line between paragraphs, and between a heading and its text.

3. Open the Pull Request with the template

After your first commit on the branch, GitHub shows a yellow banner with a **Compare & pull request** button. If the banner is gone, open the **Pull requests** tab, click **New pull request**, choose **base: main** and **compare:** your branch, and click **Create pull request**.



The PR creation form, pre-filled with the template

The description is pre-filled with our template. Fill in every section:

- **Document(s) under review:** the file names.
- **Type of review:** full or partial (see the next step).
- **Summary of changes:** what changed and why.
- **Reviewer instructions:** leave them as they are, or add anything specific.
- **Review deadline:** a date.

Add the reviewers on the right under **Reviewers**, then click **Create pull request**. See “How a review is requested” under “For reviewers” for the details.

A few minutes later the **Checks** tab shows the **Build documents** run. When it is green, the Word file is available to reviewers as the rendered-documents artifact.

4. Full review vs partial review

GitHub only lets reviewers comment on lines that are part of the PR’s changes, plus a few lines around them.

- **New document (full review).** Every line is new, so every line can be commented on. This is what happened in the example.
- **Changes to an existing document (partial review).** Only the changed lines, and a little context around them, can be commented on. This is what you want most of the time.

When you need a full review of an existing document (for example, the yearly review), use the “add the whole file” trick, so that every line is new again:

1. Create a branch from main where the document is deleted, for example review/base-sample-policy. Create the branch as in step 1, open the document on it, choose **Delete file** from the ... menu, and commit directly to that branch.
2. Create your working branch from main as usual (the document is still there), and make any changes.
3. Open the PR with **base** = review/base-sample-policy and **compare** = your working branch.
4. Every line of the document now appears as added, and reviewers can comment anywhere.
5. When the review is finished, close that PR without merging, and open a normal PR from your working branch into main to publish the result. Delete the review/base-... branch.

Tick **Partial review of changes** or **Full review of a new document** in the template so reviewers know what to expect.

5. Answer comments

Reply in the thread of each comment, even if only to say “Done”. If you disagree, explain why in the thread; the reviewer can then accept your explanation or insist.

In the example, the author answered the 48-hours comment before fixing it:

Open Add sample Information Security Policy #1
jmizquierdo wants to merge 1 commit into `main` from `docs/sample-policy`

jmizquierdo requested a review from heydolon 26 minutes ago

heydolon requested changes 3 minutes ago [View reviewed changes](#)

heydolon left a comment Collaborator ...

Thanks, the structure looks good. I've left four comments: a typo (with a suggestion you can commit directly), a vague password rule, a missing value in the classification table, and inconsistent incident-reporting deadlines. Please address them and I'll take another look.

docs/sample-policy.md

```
8 +  
9 + This policy defines the rules for protecting company information.  
10 + It applies to all employees, contractors and third parties.  
11 + The policy is reviewed annually by the Security Office.
```

heydolon 25 minutes ago Collaborator ...

Typo: "anually" should be "annually".

Suggested change

11	-	The policy is reviewed anually by the Security Office.
11	+	The policy is reviewed annually by the Security Office.

[Apply suggestion](#) [Add suggestion to batch](#)

Reply...

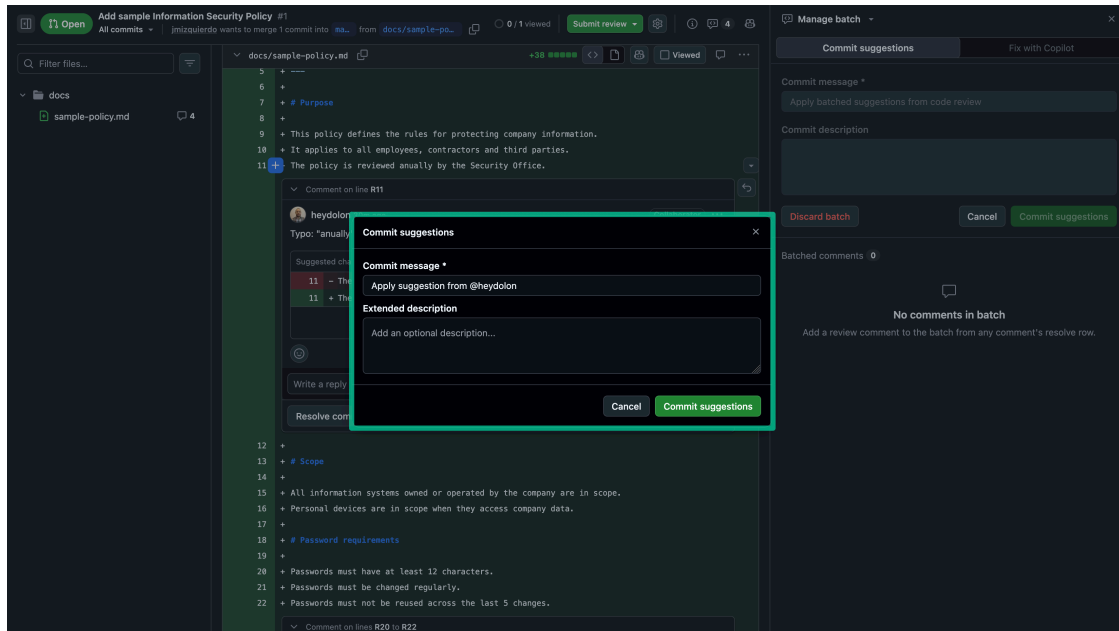
[Resolve conversation](#)

The review threads with the author's reply

6. Commit suggestions

A suggestion comment shows the proposed text with buttons under it.

One suggestion. Click **Commit suggestion** (or **Apply suggestion**), keep or edit the commit message, and click **Commit changes**. GitHub creates the commit on your branch for you, and credits the reviewer as co-author.



Committing a single suggestion

Several suggestions at once. Click **Add suggestion to batch** (or **Add to batch**) on each suggestion you accept. Then click **Commit suggestions** at the top of the page to create one commit with all of them. This is better than one commit per typo.

7. Make your other fixes

Fix the comments that weren't suggestions directly on GitHub:

1. In the PR's **Files changed** tab, open the **...** menu in the file header and choose **Edit file**. This opens the file on your branch.
2. Make the changes.
3. Click **Commit changes**, describe the fix (for example "Address review: password rotation period, Restricted encryption, critical incident deadline"), and choose **Commit directly** to your branch.

The PR updates automatically, the Word file is rebuilt, and the reviewers are notified.

8. Resolve the conversations

When a comment has been dealt with, click **Resolve conversation** at the bottom of its thread. Resolved threads collapse, so everyone can see at a glance what is still open.



Resolved threads, collapsed

Click **Show resolved** to see the whole thread again:

Open

Add sample Information Security Policy #1

jmizquierdo wants to merge 3 commits into `main` from `docs/sample-policy`

docs/sample-policy.md

Outdated

Hide resolved

29	+	Public	Website content	No	
30	+	Internal	Org charts	No	
31	+	Confidential	Customer data	Yes	
32	+	Restricted	Encryption keys		

heydolon 47 minutes ago

Collaborator

Missing value; should be Yes.

jmizquierdo 7 minutes ago

Owner

Author

Done, Restricted is now Yes.

Reply...

Unresolve conversation

jmizquierdo marked this conversation as resolved.

docs/sample-policy.md

Outdated

Hide resolved

heydolon 47 minutes ago

Collaborator

This doesn't match line 36. Normal incidents have to be reported within 24 hours, but critical ones get 48 hours. Critical incidents should have the same deadline or a shorter one. Is 48 hours a mistake?

jmizquierdo 24 minutes ago

Owner

Author

Good catch, that's a mistake. Critical incidents must be reported faster, not slower. I'll change it to 4 hours.

jmizquierdo 7 minutes ago

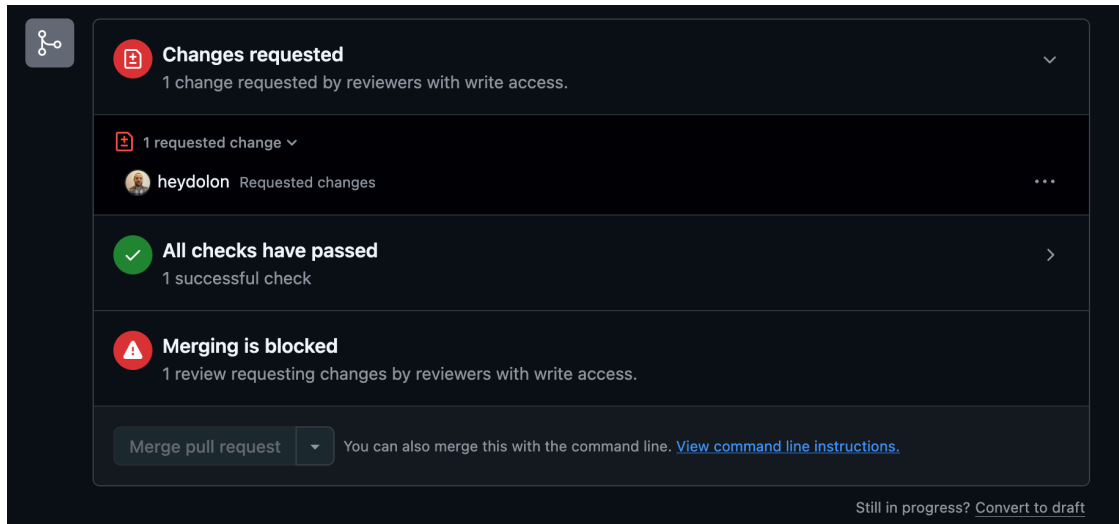
Owner

Author

Fixed: critical incidents now have a 4-hour deadline.

A resolved thread: comment, replies, and fix

main requires every conversation to be resolved before merging. Resolving is not the same as approval, though: while the reviewer's "Request changes" is in place, the PR stays blocked.

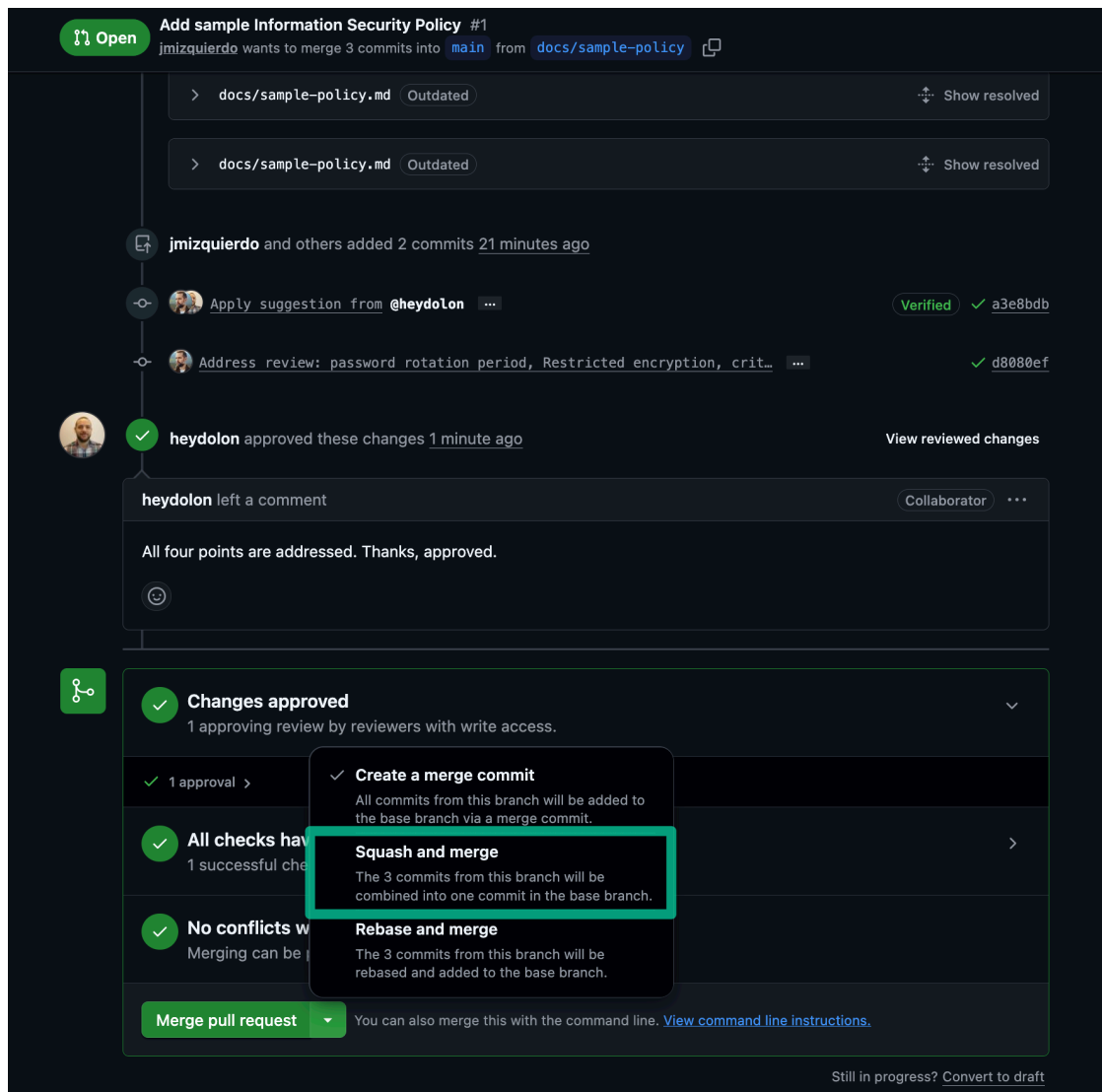


All threads resolved, but still blocked by the requested changes

Ask the reviewer to look again; they re-review only what changed (see “For reviewers”, step 8).

9. Merge

When the PR has the approval it needs, the checks are green, and every conversation is resolved, the merge box turns green. Open the arrow next to the merge button, choose **Squash and merge**, and confirm.



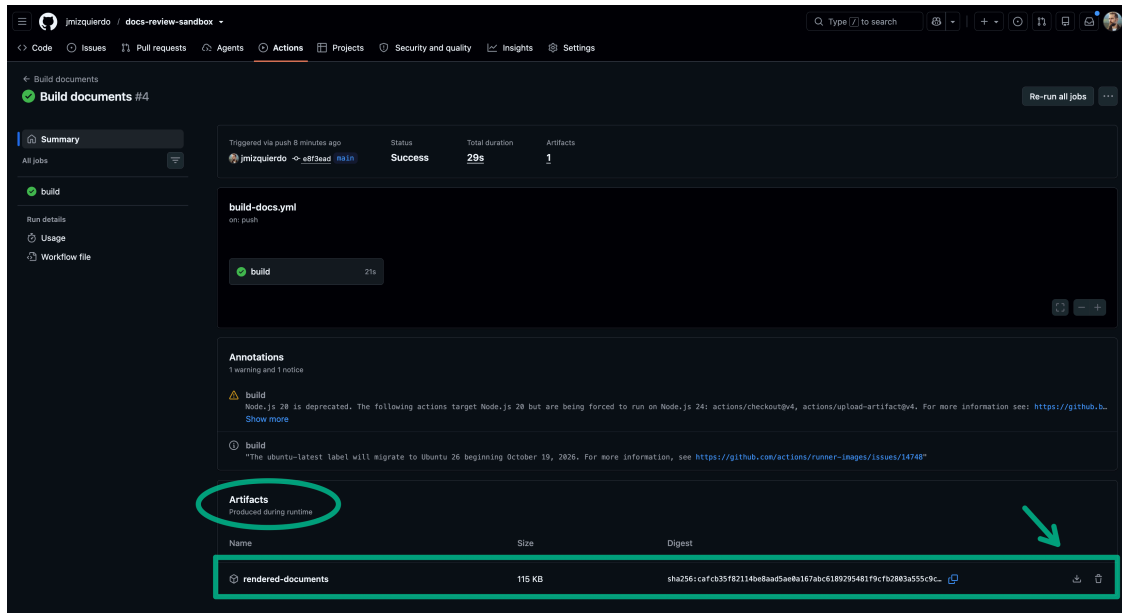
The merge box after approval, with Squash and merge selected

Squashing turns all the commits of the PR into one commit on main, so the history of the document reads as one entry per reviewed change. After merging, click **Delete branch**.

10. Get the published files

Merging into main runs **Build documents** again. To download the final, approved Word file:

1. Open the **Actions** tab of the repository.
2. Click the latest **Build documents** run on main.
3. Download rendered-documents from the **Artifacts** section.



The final rendered-documents artifact of the build on main

This is the file to publish or share. For a PDF, open it in Word and save it as PDF. Never edit the Word file by hand: any change must go through a new PR.

Artifacts are kept for 90 days by default. For long-term publishing, attach the files to a GitHub Release or copy them to the official location.

FAQ

I can't see the blue + button. You are probably in the rendered (rich diff) view. Switch to the source view with the <> icon in the file header. Also check that you are signed in and that you were added as a collaborator. Lines far from any change can't be commented on in a partial review; see "Full review vs partial review".

Can I review in Word? You can **read** the DOCX from the artifact, but comments and tracked changes in that file go nowhere. Write your comments and suggestions on the PR.

Where is the final PDF? The build only produces the Word file: **Actions** tab, latest **Build documents** run on main, **Artifacts** section, rendered-documents. Open it in Word and save it as PDF.

Why are my comments marked "Pending"? You started a review and haven't submitted it yet. Click **Submit review** and choose a verdict; until then, nobody else can see your comments.

I fixed everything but the PR is still blocked. The reviewer's "Request changes" verdict stays until the reviewer approves. Ask them to re-review.

Why can't I approve the PR? GitHub doesn't let authors approve their own PRs. Another person has to approve.

What does "Outdated" mean on a comment? The line the comment was written on has changed since. That usually means the author already acted on it.

Can the author ignore a suggestion? Yes. They should reply explaining why, then resolve the thread.

A suggestion contains an instruction instead of new text. What do I do? Don't commit it, because it would replace the line with the instruction. Do what it asks by hand, reply in the thread, and resolve it.

Do I need to install anything? No. Reviewers and authors do everything on the GitHub website.

Cheat sheet

Reviewers

I want to...	Do this
Find PRs waiting for me	Pull requests in the top bar, then Review requests
Read the formatted document	Files changed , then the Display the rich diff icon
Get the Word file	Checks , then Build documents , then Artifacts , then rendered-documents
Comment on a sentence	Source view, hover over the line number, click the blue +
Comment on several lines	Drag the + from the first to the last line
Propose new wording	In the comment box, click Add a suggestion (first toolbar icon) or press Cmd+G, then edit the text inside the block
Send all comments together	Start a review on the first comment, then Submit review at the end
Block until fixed	Submit with Request changes
Accept the document	Submit with Approve
See only what changed since my review	Files changed , All commits menu, Changes since your last review

Authors

I want to...	Do this
Start a change	Code tab, branch menu (shows main), type a name, Create branch
Write reviewable text	One sentence per line, empty line between paragraphs
Ask for a review	Open a PR, fill in the template, add reviewers
Have reviewers requested automatically	Add a line to CODEOWNERS (file pattern, then @username) through a PR
Get a full review of an existing document	The “add the whole file” trick: PR against a branch where the file is deleted
Accept one suggestion	Commit suggestion , then Commit changes

I want to...	Do this
Accept several suggestions	Add suggestion to batch on each, then Commit suggestions
Fix other comments	Files changed , file ⋯ menu, Edit file , then Commit changes to your branch
Close a comment	Reply, then Resolve conversation
Publish	After approval, Squash and merge , then Delete branch
Get the published files	Actions , latest Build documents run on main, Artifacts

Rules on main: at least 1 approval, all conversations resolved, no direct changes.